

Лабораторная работа №4. Загрузка и обработка изображений

Что вы сделаете

Вы завершите сайт сервисом `/services/images/`. Пользователь выбирает JPEG, PNG или WebP до 5 МБ, Django проверяет настоящее содержимое файла, сохраняет его в `media`, передаёт локальной модели и показывает три результата с разными по длине шкалами.

Основные темы — `request.FILES`, `ImageField`, серверная проверка загрузки, различие URL и пути файла, обработка ошибок и итоговая проверка удобства всего сайта. Код распознавания в `services.py` уже готов и не является предметом опроса.

1. Исходный проект и пакеты

Продолжите ЛР3 либо распакуйте `starter.zip` из каталога ЛР4. Starter содержит проверенное состояние после ЛР3 без базы, окружения, `media` и моделей. Работайте в каталоге с `manage.py`.

Windows и Linux с CPU:

```
python -m pip install torchvision==0.29.0 --index-url https://download.pytorch.org/whl/cpu
python -m pip install Pillow==12.3.0
```

macOS:

```
python -m pip install torchvision==0.29.0 Pillow==12.3.0
```

Проверьте:

```
python -c "import torchvision, PIL; print('Пакеты установлены')"
```

2. Подключение готовых файлов

Распакуйте содержимое `готовые_файлы.zip` из каталога ЛР4 рядом с `manage.py`. Появятся приложение `image_classifier`, шаблон и обновлённая команда подготовки моделей. `startapp` выполнять не нужно.

Назначение файлов:

- `models.py` — запись с изображением и результатом;
- `forms.py` — проверка размера, формата и содержимого;
- `views.py` — сохранение, распознавание и удаление при ошибке;
- `services.py` — готовая обёртка MobileNetV3 Small;
- `urls.py`, `tests.py` и шаблон страницы.

3. Подключение приложения и маршрута

Добавьте в `INSTALLED_APPS` файла `config/settings.py`:

```
"image_classifier",
```

В `config/urls.py` добавьте в `urlpatterns`:

```
path("services/images/", include("image_classifier.urls")),
```

В `templates/base.html` после ссылки «Диалог» добавьте:

```
<li><a href="{% url 'image_classifier:index' %}"
{% if active_page == 'images' %}aria-current="page"{% endif %}>Изображения</a></li>
```

В карточке «Классификация изображений» из `core/views.py` замените `"url_name": None` на:

```
"url_name": "image_classifier:index",
```

4. Настройка загружаемых файлов

В конец `config/settings.py` добавьте:

```
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"
DATA_UPLOAD_MAX_MEMORY_SIZE = 6 * 1024 * 1024
```

`MEDIA_ROOT` — реальный каталог на компьютере. `MEDIA_URL` — начало адреса, по которому браузер получает файл. Общий предел запроса немного больше 5 МБ, потому что форма добавляет служебные данные.

В начало `config/urls.py` добавьте импорты:

```
from django.conf import settings
from django.conf.urls.static import static
```

После списка `urlpatterns` добавьте:

```
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Так Django раздаёт загруженные файлы во время локальной разработки. Это не схема production-развёртывания, которое не входит в курс.

Убедитесь, что `.gitignore` содержит:

```
db.sqlite3
media/*
!media/.gitkeep
.venv/
```

5. Модель и миграция

Откройте `image_classifier/models.py`. `ImagePrediction` хранит:

- `session_key` для разделения истории;
- `image` как `ImageField`;
- `labels` как небольшой JSON-список результатов через совместимый `JSONListField`;
- дату создания.

`upload_to="predictions/%Y/%m/%d"` раскладывает файлы по датам внутри `media`, чтобы один каталог не разрастался.

Создайте таблицу:

```
python manage.py makemigrations image_classifier
python manage.py migrate
```

6. Как проверяется файл

Откройте `image_classifier/forms.py`. Виджет `accept` помогает диалогу выбора показать подходящие файлы, но не защищает сервер: запрос можно сформировать вручную.

Метод `clean_image()` выполняется при `form.is_valid()`:

1. Сравнивает размер с `5 * 1024 * 1024` байт.
2. Открывает поток через `Pillow`.
3. Вызывает `verify()`, проверяя структуру изображения.
4. Разрешает только реальные форматы JPEG, PNG и WebP.
5. Возвращает указатель потока в начало через `seek(0)`, чтобы Django затем сохранил весь файл.

Поэтому текстовый файл, переименованный в `photo.png`, должен быть отклонён. Проверять только расширение или присланный `Content-Type` недостаточно.

7. Как view обрабатывает POST

Откройте `image_classifier/views.py`. Обратите внимание на создание формы:

```
form = ImagePredictionForm(request.POST, request.FILES)
```

Без `request.FILES` форма не получит изображение. В HTML у формы обязательно есть:

```
enctype="multipart/form-data"
```

После проверки `view`:

1. Получает объект через `form.save(commit=False)`.
2. Добавляет ключ текущей сессии и сохраняет файл.
3. Передаёт в готовый сервис `prediction.image.path`.
4. Записывает три результата и делает `redirect`.

`image.path` — путь на диске, который нужен `Pillow`: например `C:\project\media\...\cat.jpg`.

`image.url` — адрес для браузера: `/media/...\cat.jpg`. В шаблоне используется URL, в Python-обработке — `path`.

Если распознавание завершается ошибкой, `view` удаляет и запись, и уже сохранённый файл, затем показывает понятное сообщение в форме. Иначе в `media` оставался бы мусор.

Текущий результат ищется одновременно по `pk` и `session_key`. Замена номера в строке браузера не должна открывать изображение другой сессии.

8. Готовый распознаватель

`services.py` загружает компактную `MobileNetV3 Small` и возвращает три английские подписи с процентами. Считайте его функцией:

```
путь к изображению → список из трёх результатов
```

Список классов ограничен учебным набором `ImageNet`, поэтому ответ может быть приблизительным или неожиданным. Разбираться во внутренней математике модели не требуется.

Для предварительного скачивания:

```
python manage.py prepare_models --service image
```

9. Шаблон, предпросмотр и шкалы

`templates/image_classifier/index.html` показывает ограничения до отправки, локальный предпросмотр и текущий результат. Предпросмотр добавляется `static/js/app.js`; при выключенном JavaScript загрузка и ответ сервера всё равно работают.

Ширина результата рассчитывается через `widthratio`:

```
{% widthratio item.score 100 100 as score_width %}  
<span style="width: {{ score_width }}%"></span>
```

Это не даёт русской десятичной запятой превратить значение CSS в ошибочное. Полоса 20% должна занимать примерно пятую часть, а не всю ширину.

10. Ручная проверка

Запустите сервер и откройте `<http://127.0.0.1:8000/services/images/>`:

```
python manage.py runserver
```

Проверьте по порядку:

1. Выберите обычный JPEG/PNG/WebP меньше 5 МБ — до отправки появляется предпросмотр.
2. Отправьте его — показываются изображение и три шкалы разной длины.
3. Обновите страницу — новая запись не создаётся.
4. Переименуйте `.txt` в `.png` и отправьте — форма должна показать ошибку.
5. Откройте приватное окно — история изображений там пуста.

Файлы для проверки можно взять любые свои; отдельная папка примеров не требуется.

11. Автоматические проверки

```
python manage.py check  
python manage.py makemigrations --check --dry-run  
python manage.py test
```

Тесты используют временный каталог `media` и предиктор-заглушку. Реальные веса не скачиваются. Проверяются валидная загрузка, поддельное изображение, `redirect`, сохранение результата, сессии и ширина шкалы.

12. Итоговый UX-аудит

Выполните три практические проверки всего сайта:

1. Откройте страницы на ширине около 360, 768 и 1280 px. Не должно быть горизонтальной прокрутки или наложения элементов.
2. Для каждого сервиса вызовите обычную ошибку: пустой текст, слишком длинное сообщение, неверный файл. Сообщение должно находиться рядом с действием и подсказывать исправление.
3. Дайте однопользователю без подсказок открыть любой сервис и получить результат. Зафиксируйте, где он замедлился или ошибся. Если задача выполнена без затруднений, это тоже допустимый

результат.

Выберите одно наблюдение с наибольшим влиянием. Исправьте подпись или пояснение, если это действительно нужно, и опишите решение в отчёте.

13. Если что-то не работает

- `No module named PIL` — установите `Pillow` в активное окружение.
- `no such table: image_classifier_imageprediction` — выполните миграции из пункта 5.
- После `redirect` картинка не открывается — проверьте обе настройки `media` и блок `static(...)`.
- В `MEDIA_ROOT` попал файл, но форма сообщает об отсутствии — проверьте `request.FILES` и `multipart/form-data`.
- Windows сообщает неверный путь — сервису передавайте `prediction.image.path`, не `.url`.
- Поддельный PNG принимается — должна выполняться функция `clean_image()` с `Image.open(...).verify()`.
- Все шкалы заполнены целиком — используйте `score_width` из пункта 9.
- После ошибки модели остаётся файл — в ветке `except` должны вызываться удаление изображения и объекта.

14. Контрольные вопросы

1. Зачем форме одновременно нужны `request.POST` и `request.FILES`?
2. Для чего используется `enctype="multipart/form-data"`?
3. Чем `image.path` отличается от `image.url`?
4. Почему `accept` и расширение файла не заменяют серверную проверку?
5. Зачем при ошибке обработки удалять и объект БД, и файл?
6. Как сервис не даёт открыть результат другой браузерной сессии?

15. Критерии приёмки и отчёт

Сервис доступен из меню; принимает только реальные JPEG/PNG/WebP до 5 МБ; показывает предпросмотр, изображение и три корректные шкалы; использует `image.path`; не оставляет запись при ошибке; история разделена по сессиям; все тесты проходят. Все три сервиса сайта открываются.

В итоговый DOCX/PDF включите титульные данные, цель, перечень добавленных файлов, скриншоты валидного результата и ошибки, объяснение загрузки файла и различия `path/URL`, результат итогового аудита и ответы. На защите запустите сайт, загрузите файл и покажите форму, `view`, модель и `media`-настройки.